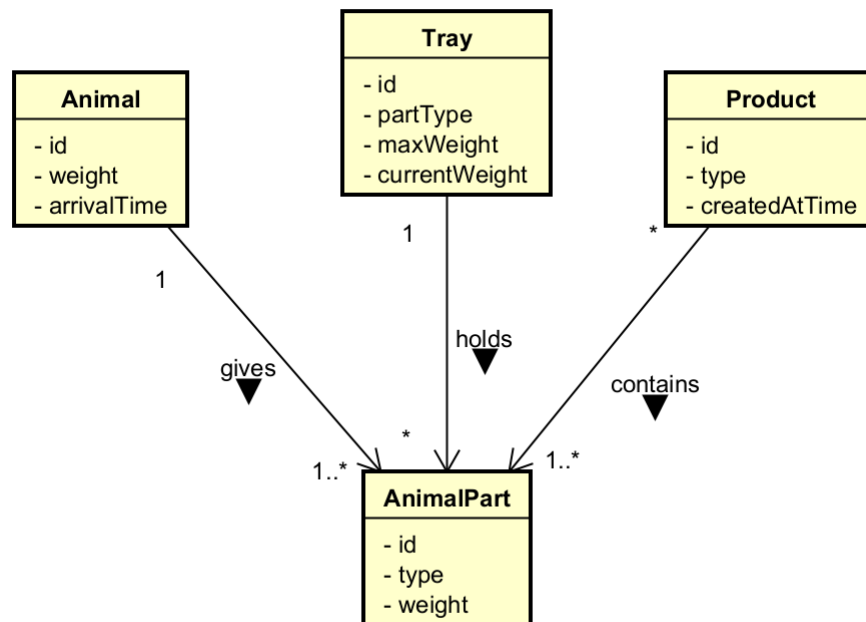


PRO3 – Assignment 1

1. Domain model



2. What the architecture must support

- Three independent stations: weighing, cutting (animals are cut into parts, which are distributed into different type trays with a weigh cap), and packing (into part-packages or “half animal” products)
- Every part references the animal it came from; every product references the trays (also references parts and animals)
- A recall must be able to start from one troublesome animal and find every product that might contain a part of it, and it must be reachable from outside the slaughterhouse
- Explicit constraint: a station must keep working even if the network is down – this is the main driver behind the design below

3. Architectural drivers

Two requirements shape almost every decision in the containers below:

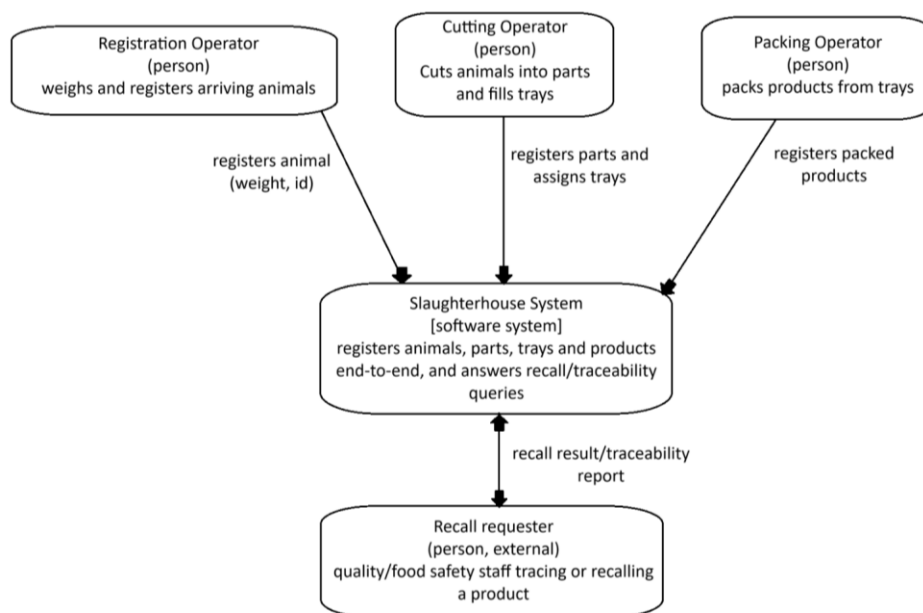
- Station autonomy: a station’s ability to register data must not depend on any other station, on a central system, or on the network being up.

- Traceable, but not real-time, recall — tracing an animal through to finished products is only needed after the fact, so the data a recall reads from can lag slightly behind the stations (eventual consistency) as long as it always catches up.

These two together point to the same architectural style: each station keeps its own local data and only talks to the rest of the system indirectly and asynchronously, while recall is served from a separate, central read model that all three stations feed.

4. C1 – System context view

The system context shows the slaughterhouse system as a single box and the people who interact with it. The three station operators register data as they work; the recall requester is explicitly outside the slaughterhouse, as required by the case.



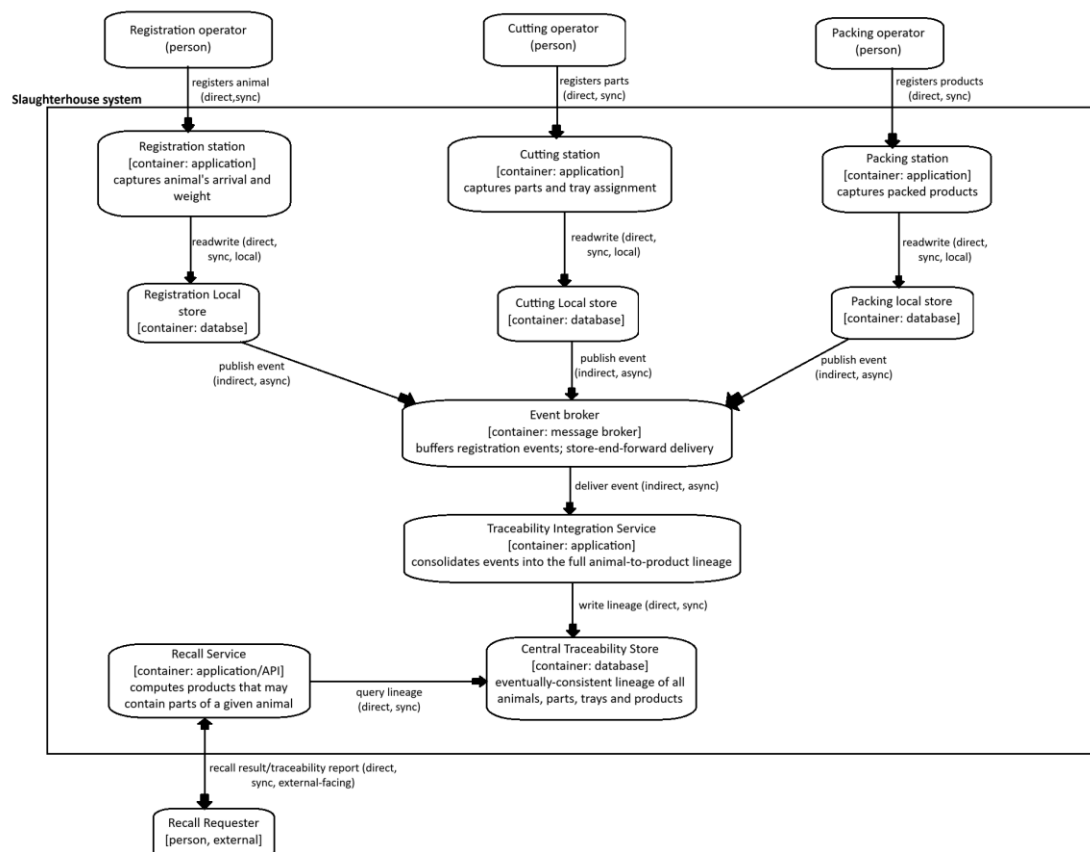
Actors

Actor	Type	Role
Registration Operator	Person, internal	Weighs and registers each arriving animal.
Cutting Operator	Person, internal	Cuts animals into parts and assigns parts to trays.

Packing Operator	Person, internal	Packs products (part packages or “half animals”) from tray contents.
Recall Requester	Person, external	Quality/food-safety staff who need to trace an animal to the products it could be part of, from outside the slaughterhouse.

5. C2 – Container view

Zooming into the system boundary, each station is its own application with its own local store, so registering an animal, a part or a product never has to wait on anything outside that station. Stations only reach the rest of the system indirectly, by publishing events to a broker that stores and forwards them whenever the network allows. A separate integration service consumes those events into one central, queryable lineage, which the external-facing Recall Service reads from.



Containers

Container	Kind	Responsibility
Registration / Cutting / Packing Station App	Application	The on-site program an operator uses at that station. Writes only to its own local store, then goes on with its work.
Registration / Cutting / Packing Local Store	Database	Local, per-station data — the source of truth for that station’s own registrations, always writable regardless of network state.
Event Broker	Message Broker	Buffers “{Animal Part Product} registered” events published by the stations and forwards them when the receiving side is reachable.
Traceability Integration Service	Application	Subscribes to the broker and folds incoming events into one consistent, cross-station lineage.
Central Traceability Store	Database	Eventually-consistent record of the full Animal → Part → Tray → Product lineage; the only place that needs data from all three stations at once.
Recall Service	Application / API	External-facing container that turns an animal id into the list of possibly-affected products, satisfying the “accessible outside the slaughterhouse” requirement.

6. Types of communication

As requested, the emphasis here is on how containers talk to each other — direct vs. indirect, synchronous vs. asynchronous — rather than on specific technologies.

Interaction	Type	Timing	Why
Operator → Station App	Direct	Synchronous	Local UI interaction at the station; not exposed to the network.
Station App → Local Store	Direct	Synchronous	Local, on-machine persistence — guarantees the station can register data with no network at all.

Station App → Event Broker	Indirect	Asynchronous	“Store-and-forward” publish. The app queues the event locally and the broker delivers it whenever connectivity returns — this is what lets a station keep working while the network is down.
Event Broker → Integration Service	Indirect	Asynchronous	Integration Service consumes events at its own pace; a slow or offline consumer never blocks a station.
Integration Service → Central Store	Direct	Synchronous	Internal write into the consolidated, queryable lineage (Animal → Part → Tray → Product).
Recall Service → Central Store	Direct	Synchronous	Internal read used to compute which products could contain parts of a given animal.
Recall Requester ↔ Recall Service	Direct	Synchronous	Exposed outside the slaughterhouse (e.g. web/API) so recalls can be requested and answered on demand.

Why this satisfies the constraint: the only thing a station ever needs to register data is its own local store — direct and synchronous, but purely local. Everything that crosses a station boundary (publishing to the broker) is indirect and asynchronous, so a network outage only delays when the central lineage catches up, never whether a station can keep registering animals, parts or products.

7. Assumptions and scope

- The domain model (Animal, Part, Tray, Product and their references) is assumed to already exist, as it has been handed in separately; this document only adds the architecture around it.
- “Event Broker”, “Local Store” and “Central Traceability Store” are communication/container roles, not product choices — concrete technologies are deliberately left open per the assignment’s instructions.
- A product such as “half an animal” can draw parts from more than one animal; the Central Traceability Store is what makes a recall query against that many-to-many lineage practical without touching the station stores directly.

- Distribution/retail systems receiving finished products are outside the system boundary and are not modelled here, since the case does not require integrating with them.